

PURPOSE OF THE RESEARCH

This document talks about how I understood the papers, how I reasoned about the problem, and how I figured out the implementation.

UNDERSTANDING Fin3R

Problem observed in feed-forward reconstruction

The feed forward 3D reconstruction models are fast and scalable as compared to structure from motion variants. They can transform multiple images into pointmaps in single forward pass with no sift or BA or any other optimization which usually sfm variants require. However, they still lag behind the monocular geometry models in capturing fine details due to which existing feed-forward 3D reconstruction approaches produce blurry geometry at the edges and corners or over-smoothed boundaries around the objects.

This happens mainly due to 2 reasons:

- 1) Lack of high-quality clean geometry dataset for training
There are very few datasets available that provide high-resolution and accurate depth and poses to train the network on.
- 2) Geometric misalignment in multi-view point map regression
Due to drift, the pose and depth errors grow with time hampering the capturing of fine details. (Pointmap regression-> It is a computer vision approach where the NN predicts a dense 3D point map directly from images, instead of outputting a depth that is a single value per pixel, the model outputs the 3D coordinate for every pixel in the camera coordinate frame.
Eg: Depthmap regression- This pixel is 3.2 meters away.
Pointmap regression- This pixel is at (1.1, -0.4, 3.2) meters wrt camera frame)

Solution idea of Fin3R

Fin3R addresses these issues through a fine-tuning strategy:

- 1) To solve this, fin3r freezes the decoder which is responsible for multi-view matching.
- 2) Only the image encoder which is responsible for feature extraction is fine tuned resulting in obtaining sharper boundaries with higher geometric accuracy without increasing test-time memory or latency.

LoRA (Low rank adaptation) adapters are tiny extra trainable layers which are added to a big neural network so that we can fine tune the entire network without changing most of the original weights. So, in our case fin3r provides LoRA adapter weights for MAST3R so it is basically loading the original master weights to which we are attaching these small lora modules in the encoder so that the model now behaves like Mast3r + fin3r.

Feature Distribution mismatch and renormalization

Fin3r uses a MoGe (monocular geometry estimator) in the encoder to distill single view depth into the encoder. This improves single-view depth accuracy but introduces few problems like:

- 1) Due to the introduction of this now the encoder features magnitudes like the dimensions changed.
- 2) But since the decoder was frozen it expects the features with a specific distribution. This mismatch degrades the multi-view matching.

To fix this, Fin3R introduces custom renormalization layers inside each LoRA block. After applying the renormalizations, features are rescaled to preserve their original norms, improving depth while maintaining multi-view consistency.

Loss formulation

Fin3R training loss has 2 components:

- 1) Monocular distillation loss -> This is responsible for aligning predicted depth with pseudo-depth from monocular geometry estimator, thereby improving single-view detail.
- 2) Pointmap regression loss -> Ensures that multi-view geometry and matching remain robust.

UNDERSTANDING MAST3R

MASt3R performs image matching directly in **3D space instead of 2D**.

It builds on DUS3R and introduces a **dense descriptor head** trained using contrastive loss, enabling pixel-accurate dense correspondences in 3D geometry.

Key contributions I noted:

- 1) Fast reciprocal matching reduces complexity from $O(w^2h^2) \rightarrow O(k \cdot wh)$
- 2) Coarse-to-fine matching enables faster convergence
- 3) Training with **metric depth** improves depth accuracy, pose estimation, and matching robustness

UAVScenes Dataset

I used the dataset: interval5_AMtown01.zip

Dataset structure:

- interval5_CAM/ → RGB images
- sampleinfos_interpolated.json → per-image metadata
- rtk_positions_raw.csv → raw RTK GPS data

Field	Usage	Purpose
OriginalImageName	File name	Pose <-> image association
SortedImageID	ordering	Temporal ordering for trajectory
T4x4	4x4 camera poses -> extrinsic	Full 6DOF ground truth

P3x3	Camera Intrinsic	For correct projection to pixel space
K1, k2, k3, p1, p2	Radial and tangential distortions	
W/H	Width and height	Image resolution

RTK data provides latitude, longitude, altitude, eastings, and northings, which can be used as ground truth for trajectory evaluation.

Phase 1 - Vanilla MAST3R vs MAST3R-FIN3R on UAVScene dataset

Goal of phase 1

To compare the performance of vanilla Mast3r with Mast3r+Fin3r on the UAVScenes dataset.

Workflow

I first ran the vanilla Mast3r inference on the UAVScenes dataset to establish a baseline.

Next task was to trace the fin3r's lora integration from vggg to create inference for mast3r. Since fin3r does not provide a direct implementation for mast3r, I referred to the vggg-fin3r inference pipeline described in the readme of fin3r repo. Specifically, I traced where the LoRA adapters were applied during inference by locating the `apply_lora` function call. It is present in `Fin3R/vggt/models/vggg.py`. This served as a reference implementation for how Fin3R expects LoRA to be configured and injected.

From this I identified 3 key details:

- 1) LoRA configuration parameters, rank, lora_alpha, target_modules, lora_dropout
- 2) Target layers for LoRA injection. From this I learnt that Fin3R applies LoRA adapters specifically to the qkv projection layers inside the attention blocks.
- 3) And instead of standard PEFT, Fin3R uses renormalized PEFT, which introduces feature renormalization to preserve the expected feature magnitude distribution for the frozen decoder. For renormalization there is no change so I imported the `get_renormalized_peft_model` function from the `vggt.models.renorm_lora` directly in my `phase1_comparison.py` code instead of creating a copy for this code for mast3r.

The VGGT architecture used in fin3r differs from mast3r, so the next step was to understand where the equivalent attention layers live in MAST3R. I explored the mast3r model structure and found the following:

- 1) `model.enc_blocks` contains a list of 24 encoder blocks
- 2) Each encoder block includes an attention module-> `block.attn`
- 3) Within each attention module, the query-key-value projection is implemented as-> `block.attn.qkv`. This linear layer is the correct insertion point for LoRA adapters in mast3r.
- 4) In vggg-fin3r implementation, LoRA adapters are applied through-> `self.aggregator.patch_embed.blocks`

In contrast, mast3r organizes its attention layers under `-> model.enc_blocks[i].attn.qkv`

Based on this I implemented a custom `apply_fin3r_lora` function for MAST3R in the `phase1_comparison.py` code. This function does the follow:

- 1) Iterates over all encoder blocks
- 2) Targets each `block.attn.qkv` layer
- 3) Applies Fin3R's LoRA configuration
- 4) Wraps the model using `get_renormalized_peft_model` to preserve feature distributions.

INFERENCE

I have 3 custom codes for phase 1 inference
Visually, 3d reconstruction and metrically

1) `phase1_comparison.py`

Purpose

This script is responsible to visually observe the quality and accuracy of depth.

This script performs the following things-

- a) Loads a fixed image pair from the UAVscenes dataset. Then the 2 frames are converted to 512 resolution as required by the model
- b) Runs the inference using vanilla mast3r by loading pretrained mast3r checkpoint to predict dense 3D pointmaps for the selected image pair.
- c) Then Applies fin3r lora adapters to qkv layers of all mast3r encoder attention blocks using renormalized PEFT wrapper by loading the fin3r-mast3r weights and reruns the inference.
- d) Visualizes and saves outputs

How to run

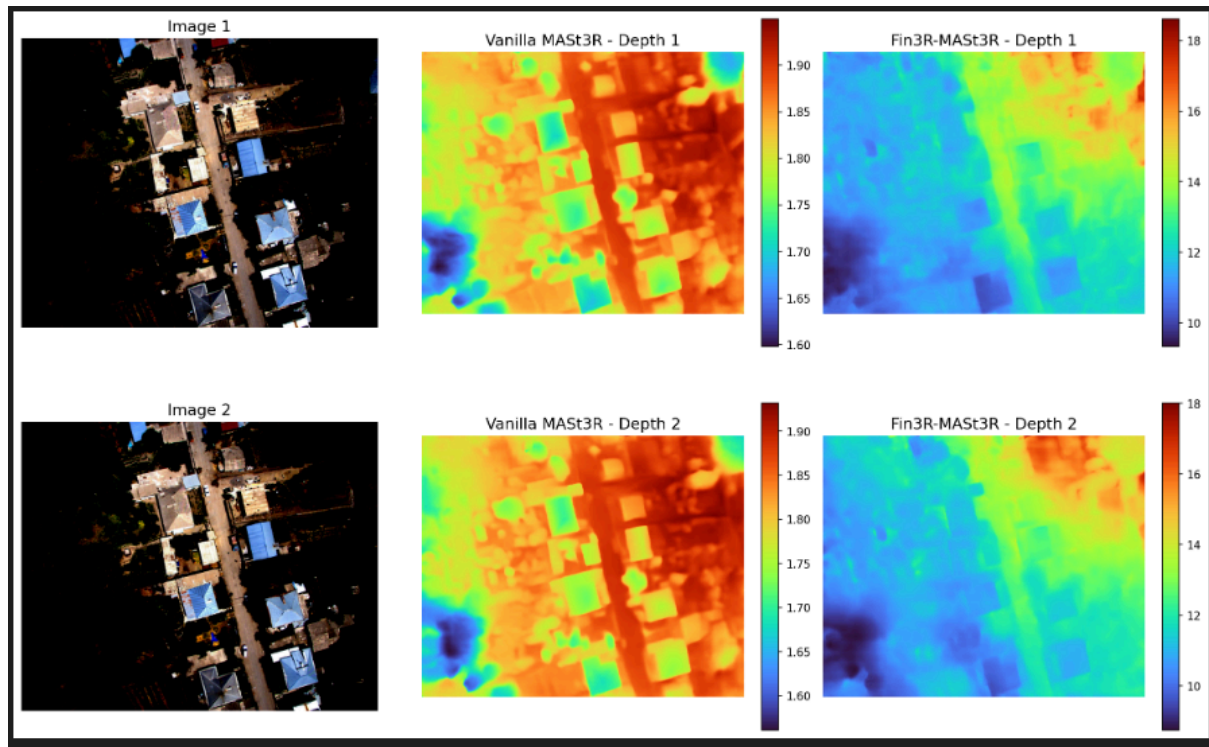
`python phase1_comparison.py`

Output location

The outputs will be stored in results folder

- a) `results/phase1/comparison_pair_0.png`
- b) `results/phase1/outputs_pair_0.pth` -> For each evaluated image pair, raw model outputs (dense 3D pointmaps, multi-view predictions, and associated metadata) were saved to disk to enable subsequent metric evaluation and reconstruction analysis without rerunning inference.

Output



The figure above shows a qualitative comparison between Vanilla Mast3r and Fin3r-Mast3r on UAVScenes dataset.

Left column -> is the input rgb images

Middle column -> is the depth maps predicted by vanilla Mast3r

Right column -> is the depth maps predicted by Fin3r-Mast3r

Depth is visualized using the z-component of the predicted 3D pointmaps, with color indicating relative distance in the camera frame.

From the visual depth comparisons:

The Vanilla Mast3r has sharper rooftop boundaries, clearer building outlines, better local contrast between structure. Basically has more visually fine details.

Whereas Fin3r-Mast3r has smoother depth fields, loss of sharp edges and corners. It is more aggressive smoothing over rooftops and roads.

So Visually, Vanilla Mast3r produces better depth on this uavscene dataset.

2) phase1_3d_reconstruction.py

Purpose

This script compares the 3d reconstruction quality of vanilla mast3r vs fin3r-mast3r. It converts predicted dense pointmap into colored 3d point cloud.

- a) Loads the rgb frames
- b) Extracts point cloud from the predicted dense pointmap. It reads pred1['pts3d'] and pred1['conf'] per-pixel confidence then flattens all pixels into a list of 3D points. Filters points using confidence threshold (conf $\geq$ 1.0). Assigns RGB colors to each retained 3D points from the corresponding input
- c) Repeats the same for Fin3R-Mast3r
- d) Creates visualizations and exports Ply files

How to run

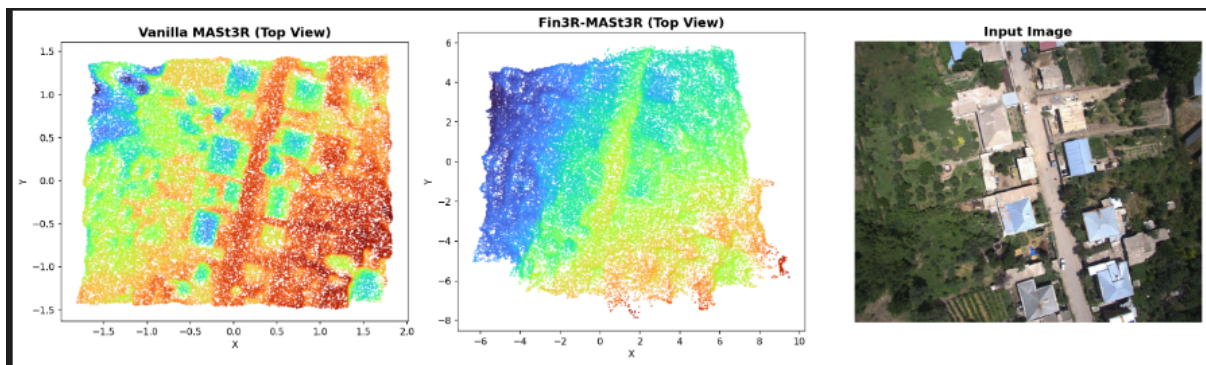
```
python phase1_3d_reconstruction.py
```

Output location

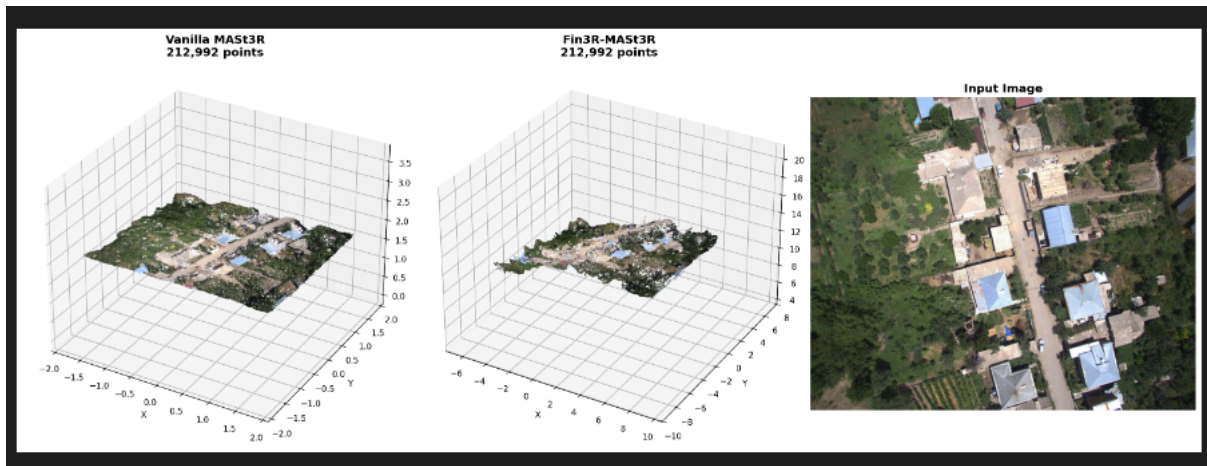
- a) results/phase1_3d_reconstruction/3d_reconstruction_comparison.png - shows the 3d reconstruction from the side view
- b) results/phase1_3d_reconstruction/3d_reconstruction_topview.png - shows 3d reconstruction from top view
- c) results/phase1_3d_reconstruction/fin3r_pointcloud.ply
- d) results/phase1_3d_reconstruction/vanilla_pointcloud.ply

Output

3d reconstruction top view: -



This is the bird's eye view of the reconstructed point clouds with depth coloring produced by vanilla mast3r and fin3r-mast3r alongside the input rgb image. It is evident that vanilla mast3r produces a compact and well aligned 3d reconstruction. It is preserving the building layout and roads whereas the fin3r-mast3r shows significantly expanded and distorted footprint and the depth quality is noticeably degraded.



Next this figure represents the point clouds for the same image. From this again it is clear that the fin3r-mast3r introduces increased smoothing and scale distortion on UAVscenes, while vanilla mast3r preserves a more stable 3d structure. Again vanilla wins.

3) phase1_lidar_simple.py

Purpose

This script performs quantitative depth comparison between vanilla mast3r and fin3r-mast3r on uavscenes by using the dataset's lidar point cloud as ground truth. The dataset provides lidar in its own sensor frame, the key challenge was to convert the lidar to per-pixel depth map in the camera frame, aligned to the image plane. This script handles that and computes standard depth error metrics like the one given in the paper

Workflow

Each frame in the dataset has both:

- i) a RGB image (present in interval5_CAM/* .jpg)
- ii) a Lidar scan (present in interval5_LIDAR/* .txt)

The script:

- a) Runs mast3r inference on the rgb image
- b) Extract predicted depth
- c) Loads lidar points and project them into the image using the calibration
- d) Convert the projected lidar into ground truth depth map
- e) Resize lidar depth map to match the scale of mast3r prediction resolution.
- f) Compute depth metrics for both vanilla and modified

The function `project_lidar_to_image_using_calib_auto()` uses camera intrinsics and extrinsics from `calibration_results.py`. I was not sure and couldn't find if the extrinsics matrix was lidar to camera or camera to lidar so my script tries both and chooses the direction that produces more valid projected pixels inside the image. Then it rasterizes a depth map using the minimum depth per pixel giving me (gt_depth, valid_mask)

Then the depth metrics are computed on valid pixels: Abs Rel, Sq Rel, RMSE, RMSE log and delta.

For computational efficiency, the script scans only the first **300 frames** of the UAVScenes sequence to identify frames that contain LiDAR data. From these, it evaluates up to **200 frames** with valid LiDAR measurements. All reported metrics are averaged over this subset.

To run:

python phase1_lidar_simple.py

Output location:

results/phase1_lidar_simple_calib/phase1_depth_metrics.json

Output

```
=====
RESULTS (median-scaled)
=====
| Method          | Abs Rel ↓ | Sq Rel ↓ | RMSE ↓ | RMSE log ↓ |  $\delta < 1.25$  ↑ |  $\delta < 1.25^2$  ↑ |  $\delta < 1.25^3$  ↑ |
|-----|-----|-----|-----|-----|-----|-----|-----|
| Vanilla MAST3R  | 0.2365    | 1.8320    | 6.714   | 0.2872    | 53.35%    | 87.07%    | 99.31%    |
| Fin3R-MASt3R   | 0.2492    | 2.1294    | 7.301   | 0.3151    | 50.01%    | 83.52%    | 97.27%    |

Evaluated on 60 frames (out of 60 with LiDAR)
Saved: results/phase1_lidar_simple_calib/phase1_depth_metrics.json
(mast3r) jitesh@jitesh-Victus-by-HP-Gaming-Laptop-15-Fa0xxx:/media/jitesh/Extremee e e Se ee e ee e e ee
e SSD/fin3r-mast3r_analysis$
```

Using LiDAR-projected depth as ground truth, we quantitatively evaluated vanilla MAST3R and Fin3R-MASt3R on UAVScenes. Depth predictions were median-scaled per frame following standard monocular depth evaluation practice. Metrics were averaged over 60 frames with valid LiDAR measurements.

Where,

- 1) Abs Rel (absolute relative error) It means on average how wrong my depth relative to the true depth(lidar). Lower is better so here vanilla mast3r has lower Abs Rel
- 2) Sq Rel (Squared Relative Error) Similar to abs rel but it punishes big error more. Like large depth errors are penalized more. Again the vanilla has lower value.
- 3) RMSE (Root mean squared error)- Basically how far off are my predictions in meters. Vanilla has 6.7m less than 7.3m for fin3r-mast3r.
- 4) RMSE log (log-scale RMSE) used for finding the relative scale mismatch. Vanilla wins
- 5) Delta metrics are the accuracy metrics. Tells what percentage of pixels are within ~25%, 56% and 95% error. So higher values are better. So again vanilla wins.

Conclusion

In this phase, I evaluated whether applying fin3r LoRA adapters to Mast3r improves performance on the UAVScenes dataset. The comparison was carried out across 3 things-> visual depth quality, 3D reconstruction, and quantitative depth metrics using Lidar as ground truth. For the quantitative evaluation, lidar point clouds were projected into the camer frame using dataset calibration, enabling per-pixel depth comparison.

It was observed that across all 3 evaluation axes vanilla mast3r consistently outperformed Fin3r-mast3r. **This outcome is likely because fin3r's LoRA adapters are trained primarily on indoor, synthetic indoor, and ground-level output datasets, whereas UAVScenes consists of aerial imagery.**

- a) TUM-RGBD (indoor RGB-D scenes),
- b) 7-Scenes (small-scale indoor environments),
- c) EuRoC (ground-based stereo-inertial MAV data in indoor/industrial settings),
- d) ETH3D SLAM (primarily ground-level outdoor and indoor scenes).

As a result, the fin3r adaptations do not transfer to aerial depth estimation.

Phase 2 - Vanilla Mast3r-slam vs Fin3r-Mast3r-slam

Goal

To evaluate whether the fin3r-mast3r encoder applied on the mast3r slam improves the slam performance on UAVScenes or not. The analysis is done by calculating slam metrics like ATE, RPE, loop closure and trajectory plots against the RTK GPS ground truth.

1) [main.py](#) - For running Vanilla Mast3r-slam

- a) Loads the UAVScenes image sequence via the mast3r-slam dataloader present at MAST3R-SLAM/mast3r_slam/[dataloader.py](#). The only modifications I did in the dataloader code is to accept .jpg format images cause earlier only .png were supported.
- b) Tracks frames, creates keyframes, and performs local/global optimization using factor graph.
- c) Saves the outputs -> trajectory, reconstruction, keyframes into the logs folder

How to run

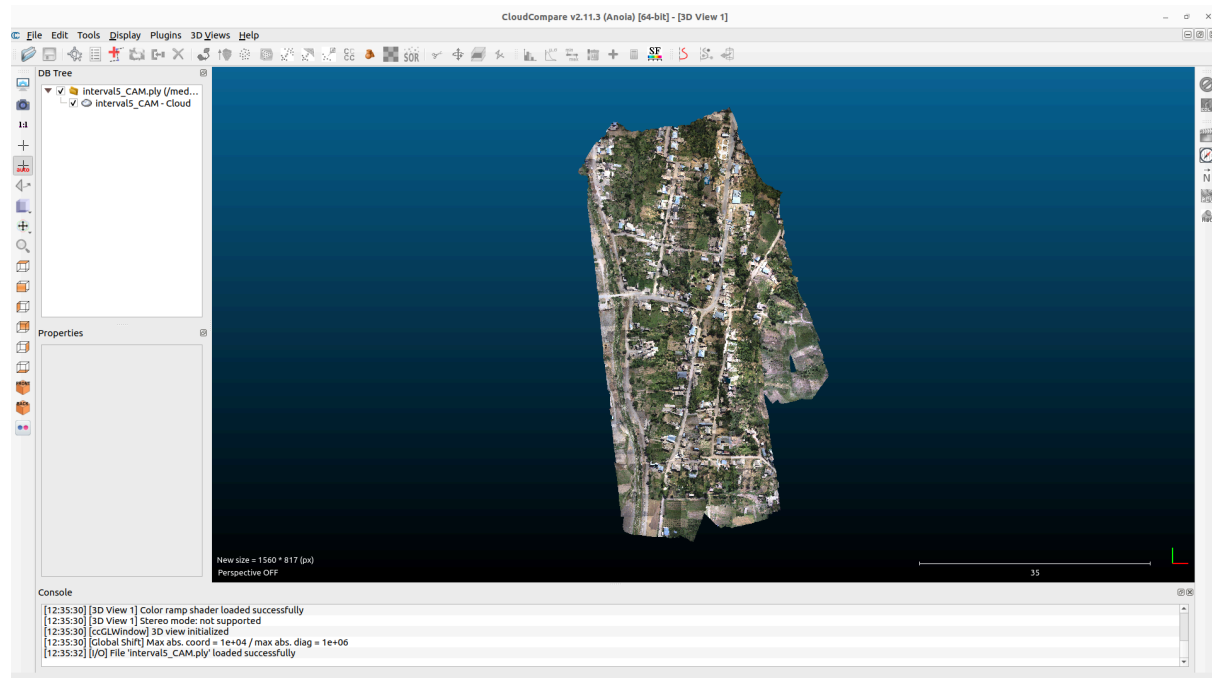
```
python main.py \
                                                    --dataset
datasets/UAVScenes_part2/interval5_CAM_LIDAR/interval5_AMtown01/interval5_CAM \
--config config/base.yaml \
--no-viz \
--save-as uavscenes_amTown01_base
```

Output Location

- a) logs/uavscenes_amTown01_base/keyframes -> keyframes are present here
- b) logs/uavscenes_amTown01_base/interval5_CAM.txt -> stores camera trajectory, trajectory is stored in TUM dataset format (t x y z qx qy qz qw)
- c) logs/uavscenes_amTown01_base/interval5_CAM.ply -> 3d reconstruction result

Output

For visualizing the 3d reconstruction I am using CloudCompare



2) main_fin3r.py - for running Fin3R-MASt3R-SLAM

- The purpose of this custom version of [main.py](#) is where I swap the encoder features by applying fin3r lora adapters on top of the mast3r encoder, then running slam with these modified features.
- I have added custom flags to this code -> `--use-fin3r` and `--fin3r-lora <path>`
- The swapping of encoder from mast3r to mast3r-fin3r is the same as done in the phase 1. The only change is that here I applied the lora weights on the backend as well because in phase 1 it was single threaded inference but in phase 2 it is multi-threaded slam so need to apply lora twice.

Step 1: Load Base Model and Share Memory

The base MASt3R model (without LoRA) is loaded and its memory is shared between processes. This allows both processes to access the same underlying weights efficiently.

Step 2: Start Backend Process

The backend process is started with flags indicating whether to apply Fin3R. The process receives the base model and will apply LoRA internally.

Step 3: Apply LoRA in Backend

Inside the backend process (`run_backend` function), if the `--use-fin3r` flag is enabled, the LoRA adapter is applied to the model before creating the factor graph and retrieval database.

Step 4: Apply LoRA in Main Process

After the backend is started, the main process applies the same LoRA transformation to its copy of the model before creating the frame tracker.

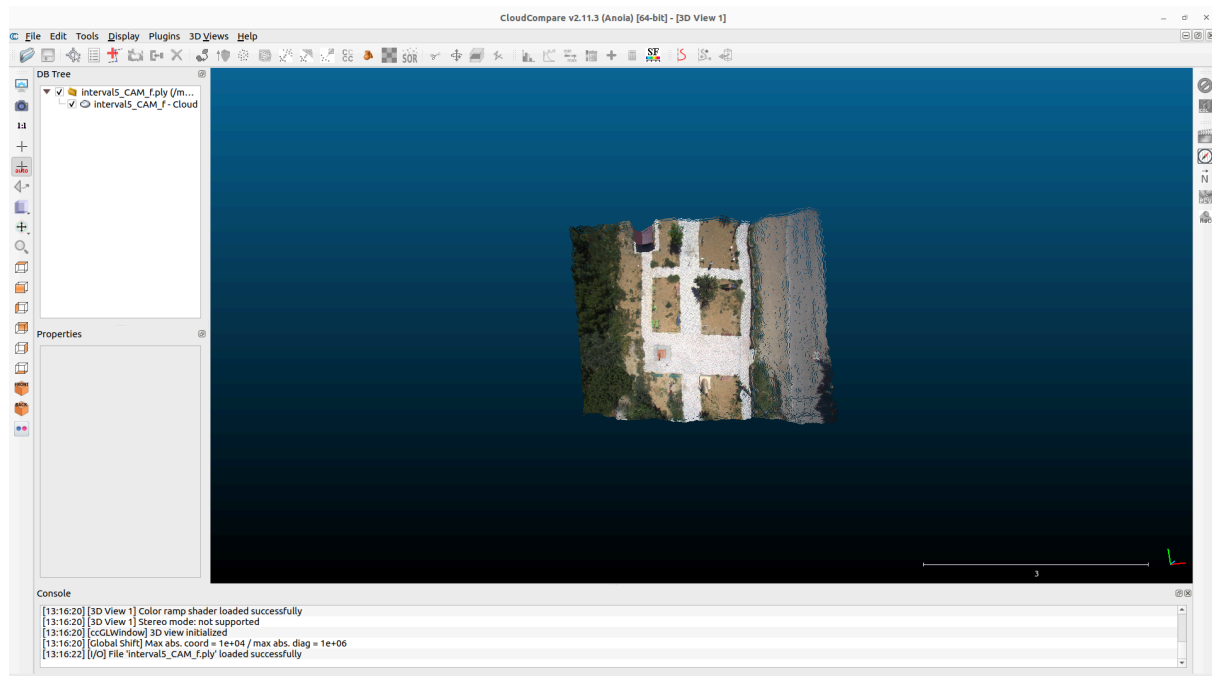
How to run

```
python main_fin3r.py \
--dataset
datasets/UAVScenes_part2/interval5_CAM_LIDAR/interval5_AMtown01/interval5_CAM \
--config config/base.yaml \
--no-viz \
--save-as uavscenes_amTown01_fin3r \
--use-fin3r \
--fin3r-lora Fin3R/checkpoints/mast3r_lora.pth
```

Output Location

- a) logs/fin3r_mast3r/keyframes -> keyframes are present here
- b) logs/fin3r_mast3r/interval5_CAM.txt -> stores camera trajectory, trajectory is stored in TUM dataset format (t x y z qx qy qz qw)
- c) logs/fin3r_mast3r/interval5_CAM.ply -> 3d reconstruction result

Output



The tracking in this was lost after 5-7 frames from the beginning so the slam was only able to reconstruct using the 5-7 initial frames. Explained in detail below while comparing the trajectory with ground truth.

3) align_uav_rtk.py

Purpose

This script is used to compare slam trajectories against rtk ground-truth trajectories on the uavscenes dataset.

Mast3r-slam produces trajectories in some arbitrary coordinate frame and scale, while uavscenes provides the ground truth in metric RTK GPS in easting-northing reference frame. Because of this mismatch we cannot compare the predicted trajectory with the ground truth.

The script's goal is to:-

- a) Bring both the predicted and the gps trajectories into a common local 2d frame
- b) Do correction for orientation and scale differences.
- c) And calculate slam evaluation metric - ATE, RPE, drift and loop closure

Workflow

- a) Load the ground truth and predicted trajectory obtained after running [main.py](#) and `main_fin3r.py` respectively for vanilla vs fin3r-mast3r. The trajectory are stored in TUM dataset format i.e. (t x y z qx qy qz qw)
- b) Next is the alignment. The slam trajectory is expressed in a local camera-centric coordinate system, while rtk is in a global frame. To make the 2 trajectories comparable: I mirrored the slam trajectory first about y-axis and then rotated it 120 degrees anticlockwise so that its direction of travel matches that of RTK trajectory.
- c) Normalize origins and estimating scale: Now because monocular slam does not recover absolute scale: The ground truth that is the RTK trajectory is zeroed at its first measurement by subtracting the station easting and northing from all the easting and northing data so that trajectory starts from (0, 0). And next the slam trajectory is also shifted to start at the origin.

Now to convert the slam trajectory into metric units to compare with the ground truth trajectory I had to calculate the scale factor. For this first I calculated the total path length for both the trajectories the predicted (slam) and ground truth (gps) and:

$$\text{Scale} = (\text{RTK path length}) / (\text{SLAM path length})$$

This is then used to convert slam trajectory to metric units.

In summary:

After alignment and scaling:

- The SLAM trajectory is:
 - Oriented correctly
 - Scaled to metric units
 - Zeroed at the origin
- The RTK trajectory is:
 - Zeroed at the origin
 - Time-normalized

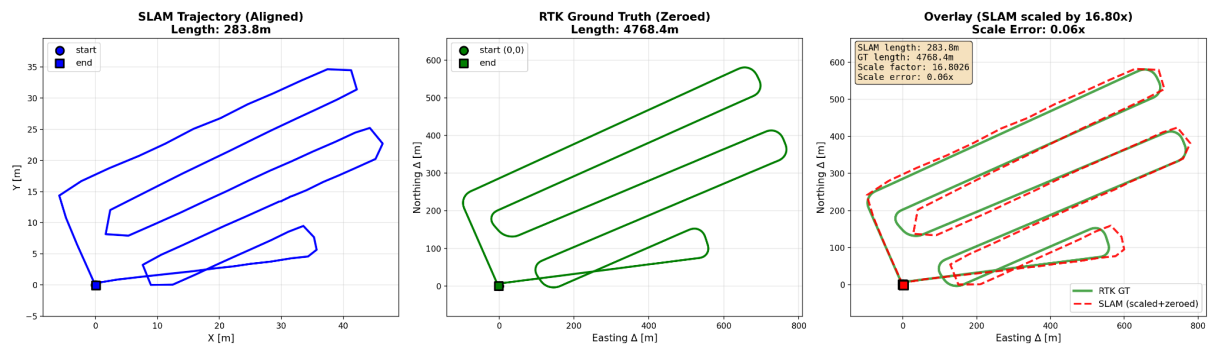
How to run

```
python align_uav_rtk.py --slam logs/uavscenes_amTown01_base/interval5_CAM.txt --rtk
datasets/UAVScenes_part2/interval5_CAM_LIDAR/interval5_AMtown01/rtk_positions_raw.c
sv --slam-out logs/uavscenes_amTown01_base/slam_vanilla_scaled.txt --rtk-out
logs/uavscenes_amTown01_base/rtk_gt_zeroed.txt --plot-out
logs/uavscenes_amTown01_base/alignment_fixed.png
```

Output

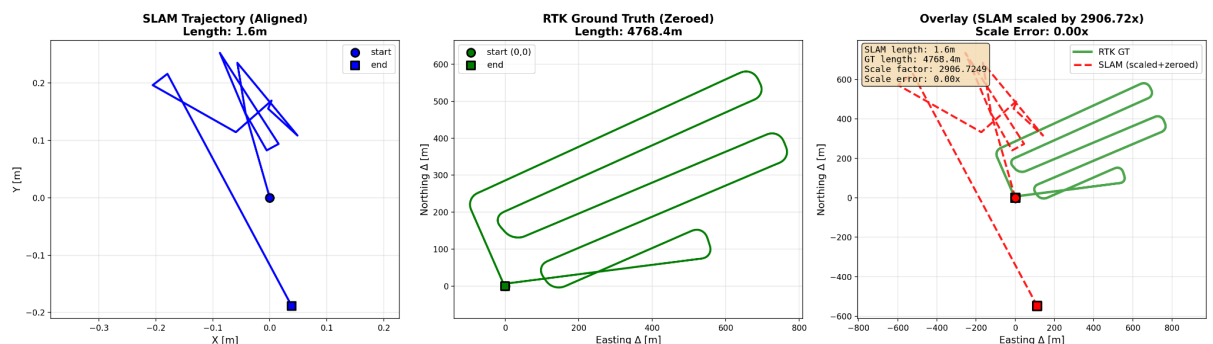
a) Vanilla Mast3r slam trajectory vs ground truth (top down view)

Location: - logs/uavscenes_amTown01_base/alignment_fixed.png



This figure shows the comparison between the slam trajectory and the RTK gps trajectory. As calculated the scale correction factor came out to be 16.8x. After scaling the slam trajectory closely follows the ground truth. On the left in blue is the slam trajectory which is unscaled. In the middle in green is the RTK gps trajectory after zeroing. On the right is the comparison after scaling the slam trajectory. This proves that just the vanilla mast3r slam on the uavscene dataset is performing decent tracking.

b) Fin3r-Mast3r slam trajectory vs ground truth (top down view)



This figure shows the fin3r-mast3r slam vs RTK. From the trajectory itself it is evident that this has severe tracking degradation. The slam system fails to accumulate consistent motion over time, leading to collapse of the estimated trajectory. This

reason is because it was only able to track for the initial few frames and then completely went in the relocalisation loop for rest of the keyframes.

Atleast visually it is evident that the Fin3r addition on mast3r do not improve slam performance for aerial datasets.

4) rescale_slam_timestamps.py

This is a helper code which adjusts the temporal scale of the slam trajectory so that its duration matches the duration of the ground truth trajectory. This step is required because monocular slam systems often produce trajectories with arbitrary or inconsistent timestamps, which prevents meaningful computation of time dependent slam metrics like RPE or drift. So, earlier we did spatial alignment and scaling now this code address temporal alignment.

This generates the `slam_vanilla_scaled_timefix.txt` new slam trajectory which temporal alignment with gps at `logs/uavscens_amTown01_base` for the vanilla and similarly for the `fin3r-mast3r` in the logs folder itself.

How to run

```
python                                rescale_slam_timestamps.py
logs/uavscenes_amTown01_base/slam_vanilla_scaled.txt
logs/uavscenes_amTown01_base/rtk_gt_zeroed.txt
logs/uavscenes_amTown01_base/slam_vanilla_scaled_timefix.txt
```

Output

```
(mast3r-slam) root@bf6e9ca6f814:/workspace/MASt3R-SLAM# python rescale_slam_time
stamps.py      logs/fin3r_mast3r/slam_vanilla_scaled.txt      logs/fin3r_mast3r/rt
k_gt_zeroed.txt      logs/fin3r_mast3r/slam_fin3r_scaled_timefix.txt
TIMESTAMP RESCALING

SLAM duration: 86.10 s (14 poses)
GT duration:   1354.34 s (6773 poses)
Scale factor:  15.7298x

Rescaled timestamps:
  Original: 0.00 → 86.10 s
  New:      0.00 → 1354.34 s

[Saved] logs/fin3r_mast3r/slam_fin3r_scaled_timefix.txt
(mast3r-slam) root@bf6e9ca6f814:/workspace/MASt3R-SLAM#
```

This is the output after running the temporal alignment for fin3r-mast3r slam we will get Similar format result for the vanilla as well. The output is located - `logs/fin3r_mast3r/slam_fin3r_scaled_timefix.txt`

4) evaluate_slam_metrics.py

This script evaluates an estimated slam trajectory against a ground-truth trajectory and compares them on slam metrics.

ATE measures the global position accuracy. For each aligned timestamp, it computes the euclidean distance between the estimated position and the Ground truth positions

RPE measures the local motion consistency. Basically it compares the relative motion between current pose and next pose in both translation and rotation. In our case I am only calculating translational RPE, the rotational RPE is null.

Drift -> Drift is simply the error accumulation over the run. It computes the cumulative distance traveled. It measures the final position error and normalizes it by 2 things - total distance -> that is drift per meter and total time -> drift per sec.

Loop closure detection

To detect a loop closure, the system looks at the **ground-truth trajectory** and checks whether the robot or drone **returns close to a place it visited earlier**. At each moment along the path, the current position is compared against **all previously visited positions** that are far enough in time to avoid trivial matches. If the current position is found to be within a small distance (for example, 5 meters) of any earlier position, this is considered a loop closure. For each detected loop, the script compares how close the estimated trajectory returns relative to ground truth and records the difference as error. So output is 2 things - number of detected loops and loop closure error.

Output

A) Vanilla Mast3r-slam trajectory vs ground truth gps trajectory

Below is the evaluation comparison the estimated SLAM trajectory against RTK ground truth using standard SLAM metrics: **Absolute Trajectory Error (ATE)**, **Relative Pose Error (RPE)**, **drift**, and **loop closure error**. The goal is to assess both **global accuracy** and **local motion consistency** over a long aerial trajectory.

```
=====
SLAM TRAJECTORY EVALUATION RESULTS
=====

ABSOLUTE TRAJECTORY ERROR (ATE)
-----
RMSE:           83.943594 m
Mean Error:     69.954923 m
Median Error:   77.423243 m
Std Deviation:  46.398661 m
Min Error:      0.000000 m
Max Error:      160.097515 m

RELATIVE POSE ERROR (RPE)
-----
Translation RMSE: 28.221920 m
Translation Mean: 14.769093 m
Translation Med:  3.968927 m
Translation Std:  24.048923 m
Rotation:         N/A (identity quaternions)

DRIFT ANALYSIS
-----
Drift per meter:  0.000537 m/m
Drift percentage: 0.0537%
Drift per second: 0.001851 m/s
Total distance:   4668.09 m
Total time:       1354.34 s
Final error:      2.5072 m

LOOP CLOSURE ANALYSIS
-----
Number of loops:  22
Mean error:       73.2835 m
Median error:     64.8681 m
Max error:        188.4003 m
=====
```

ATE

Observed results: RMSE $\approx$ 84 m Mean error $\approx$ 70 m Median error $\approx$ 77 m Maximum error exceeds 160 m Inference: The SLAM trajectory diverges significantly from ground truth over time. Errors grow especially in later parts of the trajectory, as seen in the ATE-over-time plot.

This indicates poor global consistency and accumulated drift. The wide spread in the error histogram confirms that errors are not localized but persist throughout the sequence.

RPE

Observed results: Translation RMSE ≈ 28 m Median ≈ 4 m Large spikes visible in the RPE-over-time plot Inference: Most frame-to-frame motions are estimated reasonably well. However, occasional large spikes indicate tracking instability or feature mismatches. These spikes compound over time, contributing to global drift.

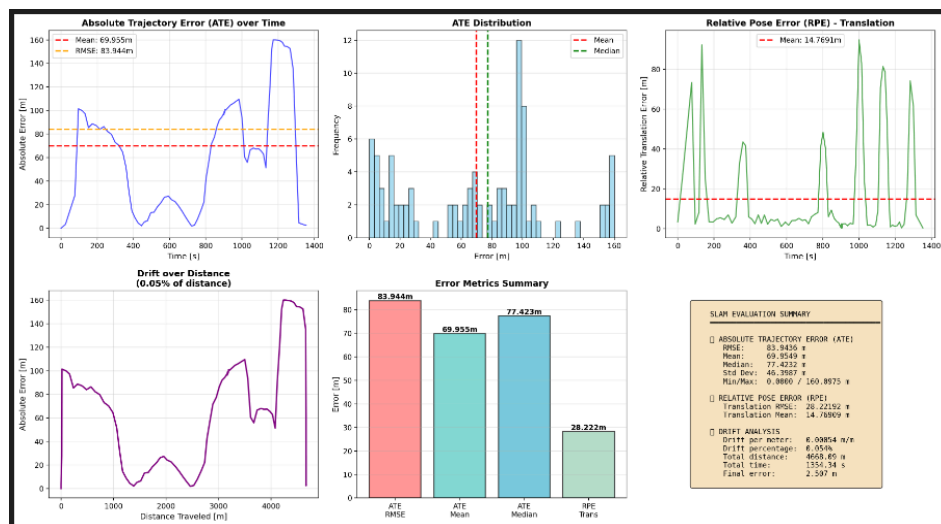
Drift Analysis

Observed results: Drift $\approx 0.05\%$ of total distance Total distance ≈ 4.7 km Final positional error ≈ 2.5 m Inference: Despite large global misalignment (ATE), the relative drift rate is low. This means the system does not explode catastrophically, but small local errors accumulate gradually. The drift-over-distance plot shows error increasing smoothly rather than abruptly.

Loop Closure Analysis

Observed results: 22 loop closures detected Mean loop error ≈ 73 m Maximum loop error ≈ 188 m Inference: Loop closures are detected, but their corrections are inaccurate. High loop errors indicate that the system recognizes revisits but fails to realign the trajectory properly. This is typical in aerial scenarios where viewpoints change significantly and appearance-based matching is weak.

Below is the graphical representation of the metrics. Not necessarily useful because from the metric itself the inference can be made. I just plotted them to make the report a bit research oriented, but it's just repetition of our inference.



B) Vanilla Fin3r-Mast3r-slam trajectory vs ground truth gps trajectory

```
=====
SLAM TRAJECTORY EVALUATION RESULTS
=====

ABSOLUTE TRAJECTORY ERROR (ATE)
-----
RMSE:          533.946026 m
Mean Error:    481.910120 m
Median Error:  447.123083 m
Std Deviation: 229.915190 m
Min Error:     0.000000 m
Max Error:     826.142922 m

RELATIVE POSE ERROR (RPE)
-----
Translation RMSE: 488.339559 m
Translation Mean: 366.806528 m
Translation Med:  266.460846 m
Translation Std:  322.379429 m
Rotation:         N/A (identity quaternions)

DRIFT ANALYSIS
-----
Drift per meter:  1111.094664 m/m
Drift percentage: 111109.4664%
Drift per second: 0.413542 m/s
Total distance:   0.50 m
Total time:       1354.34 s
Final error:      560.0761 m

LOOP CLOSURE ANALYSIS
-----
No loop closures detected
=====
```

ATE

Key values ATE RMSE: ~534 m Mean error: ~482 m Median error: ~447 m Max error: ~826 m Here, the errors are hundreds of meters, while the real trajectory length is on the order of a few kilometers. This means: The SLAM trajectory does not stay spatially close to the true path. Errors grow very large and remain large throughout the sequence. The system loses global consistency early and never recovers.

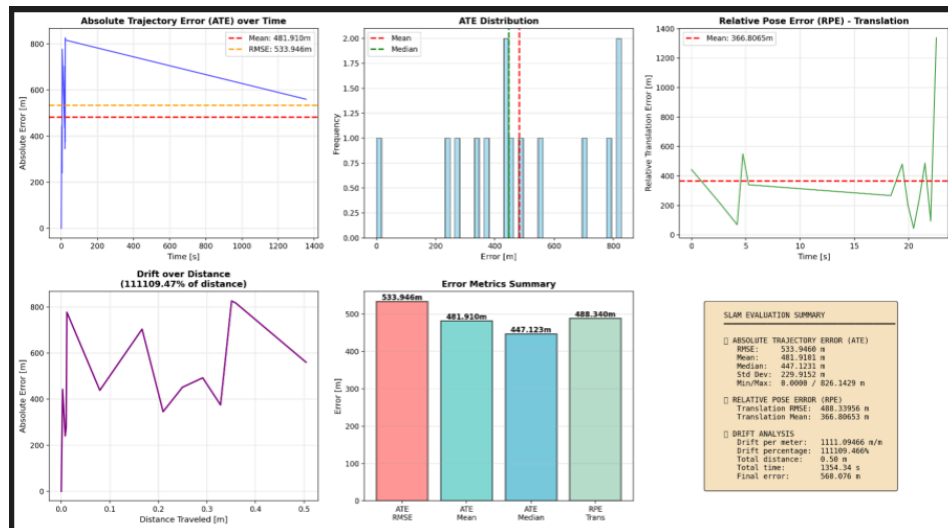
RPE

Key values Translation RMSE: ~488 m Mean translation error: ~367 m These values being extremely large means: Even frame-to-frame motion estimates are unreliable The SLAM system fails at local motion estimation.

Drift

Key values Drift per meter: ~1111 m / m Drift percentage: ~111,109% Total distance (estimated): ~0.5 m Final error: ~560 m The SLAM system believes it has traveled only ~0.5 m In reality, the vehicle traveled several kilometers This leads to an absurdly large drift percentage The system has lost any meaningful notion of scale

Loop closure: - No loop closures detected



My Final Conclusion and Verdict

This work evaluated the impact of integrating the Fin3R LoRA-enhanced MAST3R encoder into both **depth estimation** and **full SLAM** pipelines on the UAVScenes aerial dataset.

Phase 1: Depth and 3D Reconstruction

In Phase 1, Fin3R-MASt3R was evaluated against vanilla MAST3R using LiDAR-based ground-truth depth. Fin3R produced **worse local depth details** in individual frames, quantitative evaluation across standard depth metrics (Abs Rel, Sq Rel, RMSE, RMSE log, and δ thresholds) consistently showed **worse performance** than vanilla MAST3R.

These results were consistent across **more than 2,000 evaluated images**, indicating that the degradation is systematic rather than incidental.

Phase 2: SLAM Performance

In Phase 2, the Fin3R-enhanced encoder was integrated into MAST3R-SLAM and evaluated using RTK ground-truth trajectories. Compared to vanilla MAST3R-SLAM, Fin3R-MASt3R resulted in:

- Very large Absolute Trajectory Errors (ATE)
- High Relative Pose Errors (RPE)
- Severe scale inconsistency and drift
- Failure to detect loop closures

Overall, the SLAM system became unstable, frequently losing tracking and failing to maintain a coherent trajectory over long distances.

Although MAST3R and Fin3R are trained on largely similar datasets, the LoRA adaptation in Fin3R clearly changes how the encoder behaves in practice. From what I observed, Fin3R becomes much more focused on extracting fine monocular geometric details, which works well in indoor and ground-level scenes where depth ranges are limited and the structure is more regular.

That added specialization seems to help capture sharper local geometry in those settings, but it comes at the cost of generalization. When the same features are used on aerial data — where the camera sees large depth ranges, top-down viewpoints, repetitive patterns, and relatively weak parallax — the representations are no longer as reliable. In particular, they struggle with consistent frame-to-frame matching, stable scale estimation, and maintaining long-term geometric consistency.

Vanilla MAST3R, on the other hand, appears to stay more general despite being trained on similar data. Even though it may not capture the same level of fine detail in single frames, its features transfer better to aerial scenes and result in more stable behavior overall.